```
    if (City == null)
    {
        logger.LogWarning(404, "City \"{name}\" not found", Name);
        return NotFound();
    }
    logger.LogInformation("City \"{name}\"  found", Name);
    return Page();
}
```

❶ Inject the ILoggerFactory into the constructor.

❷ Use the CreateLogger method to instantiate a logger with a custom category name.

When you navigate to the City page, the log output differs in view of the fact that your customized category name is used:

```
info: City Page OnGetAsync[0]
      City "amsterdam"  found
```

This approach can be useful if you want to create logger categories at the method level, for example, but otherwise, you will most likely use an `ILogger<T>` within your application.

12.3.4 Using EventIds

Event IDs provide another way to filter log messages. For example, you might want to filter all logs that are recorded when a page cannot be found, so if necessary, you can redirect the user (and search engines) to a new permanent address. Accordingly, you might decide to associate warning logs for missing pages with an `EventId` of `404`. In this case, you pass an `EventId` in as the first parameter to one of the `Log*` methods. The `EventId` type is a struct, so you can associate an `EventId` with a log as follows:

```
_logger.LogWarning(new EventId(404), "City \"{name}\" not found", Name);
```